P2723R0
Zero-initialize objects of automatic storage duration

Published Proposal, 2022-11-15

This version:
http://wg21.link/P2723

Author:
JF Bastien (Woven Planet)

Audience:
EWG, SG12, SG22

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:
github.com/jfbastien/papers/blob/master/source/P2723r0.bs

Table of Contents

References

Informative References

§ 1. Summary

Reading uninitialized values is undefined behavior which leads to well-known security issues [CWE-457], from information leaks to attacker-controlled values being used to control execution, leading to an exploit. This can occur when:

- The compiler re-uses stack slots, and a value is used uninitialized;
- The compiler re-uses a register, and a value is used uninitialized;
- Stack structs / arrays / unions with padding are copied; and
- A heap-allocated value isn't initialized before being used.

This proposal only addresses the above issues when they occur for automatic storage duration objects (on the stack). The proposal does not tackle objects with dynamic object duration. Objects with static or thread local duration are not problematic because they are already zero-initialized if they cannot be constant initialized, even if they are non-trivial and have a constructor execute after this initialization.

```
static char global[128];            // already zero-initialized
thread_local int thread_state;      // already zero-initialized

int main() {
  char buffer[128];                 // not zero-initialized before this

  struct {
    char type;
    int payload;
  } padded;                         // padding zero-initialized with th

  union {
    char small;
    int big;
  } lopsided = '?';                 // padding zero-initialized with th

  int size = getSize();
  char vla[size];                   // in C code, VLAs are zero-initial
```

```
    char *allocated = malloc(128);     // unchanged
    int *new_int = new int;            // unchanged
    char *new_arr = new char[128];     // unchanged
  }
```

This proposal therefore transforms some runtime undefined behavior into well-defined behavior.

Adopting this change would mitigate or extinguish around 10% of exploits against security-relevant codebases (see numbers from [AndroidSec] and [InitAll] for a representative sample, or [KCC] for a long list).

We propose to zero-initialize all objects of automatic storage duration, making C++ safer by default.

This was implemented as an opt-in compiler flag in 2018 for LLVM [LLVMReview] and MSVC [WinKernel], and 2021 in for GCC [GCCReview]. Since then it has been deployed to:

- The OS of every desktop, laptop, and smartphone that you own;
- The web browser you're using to read this paper;
- Many kernel extensions and userspace program in your laptop and smartphone; and
- Likely to your favorite videogame console.

Unless of course you like esoteric devices.

The above codebases contain many millions of lines of code of C, C++, Objective-C, and Objective-C++.

The performance impact is negligible (less that 0.5% regression) to slightly positive (that is, some code gets *faster* by up to 1%). The code size impact is negligible (smaller than 0.5%). Compile-time regressions are negligible. Were overheads to matter for particular coding patterns, compilers would be able to obviate most of them.

The only significant performance/code regressions are when code has very large automatic storage duration objects. We provide an attribute to opt-out of zero-initialization of objects of automatic storage duration. We then expect that programmer can audit their code for this attribute, and ensure that the unsafe subset of C++ is used in a safe manner.

This change was not possible 30 years ago because optimizations simply were not as good as they are today, and the costs were too high. The costs are now negligible. We provide details in § 5 Performance.

Overall, this proposal is a targeted fix which stands on its own, but should be considered as part of a wider push for type and resource safe C++ such as in [P2687r0].

§ 2. Opting out

In deploying this scheme, we've found that some codebases see unacceptable regressions because they allocate large values on the stack and see performance / size regressions. We therefore propose standardizing an attribute which denotes intent, `[[uninitialized]]`. When put on automatic variables, no initialization is performed. The programmer is then responsible for what happens because they are back to C++23 behavior. This makes C++ safe by default for this class of bugs, and provides an escape hatch when the safety is too onerous.

An `[[uninitialized]]` attribute was proposed in [P0632r0].

As a quality of implementation tool, and to ease transitions, some implementations might choose to diagnose on large stack values that might benefit from this attribute. We caution implementations to avoid doing so in their front-end, but rather to do this as a post-optimization annotation to reduce noise. This was done in LLVM [AutoInitSummary].

§ 3. Security details

What exactly is the security issues?

Currently, uninitialized stack variables are Undefined Behavior if they are used. The typical outcome is that the program reads stale stack or register values, that is whatever previous value happened to be compiled to reside in the same stack slot or register. This leads to bugs. Here are the potential outcomes:

- *Benign*: in the best case the program causes an unexpected result.
- *Exploit*: in the worst case it leads to an exploit. There are three outcomes that can lead to an exploit:
- *Read primitive*: this exploit could be formed from reading leaked secrets that reside on the stack or in registers, and using this information to exploit another bug. For example, leaking an important address to defeat ASLR.
- *Write primitive*: an uninitialized value is used to perform a write to an arbitrary address. Such a write can be transformed into an execute primitive. For example a stack slot is used on a particular execution path to determine the address of a write. The attack can control the previous

stack slot's content, and therefore control where the write occurs. Similarly, an attacker could control which value is written, but not where it is written.

- *Execute primitive*: an unitialized value is used to call a function.

Here are a few examples:

```
int get_hw_address(struct device *dev, struct user *usr) {
  unsigned char addr[MAX_ADDR_LEN];         // leak this memory 😢

  if (!dev->has_address)
    return -EOPNOTSUPP;

  dev->get_hw_address(addr);                // if this doesn't fill add
  return copy_out(usr, addr, sizeof(addr)); // copies all
}
```

Example from [LinuxInfoleak]. What can this leak? ASLR information, or anything from another process. ASLR leak can enable Return Oriented Programming (ROP) or make an arbitrary write effective (e.g. figure out where high-value data structures are). Even padding could leak secrets.

```
int queue_manage() {
  struct async_request *backlog;              // uninitialized

  if (engine->state == IDLE)                  // usually true
    backlog = get_backlog(&engine->queue);

  if (backlog)
    backlog->complete(backlog, -EINPROGRESS); // oops 🔥🔥🔥

  return 0;
}
```

Example from [LinuxUse]. The attacker can control the value of backlog by making a previous function call, and ensuring the same stack slot gets reused.

Security-minded folks think this is a good idea. The Microsoft Windows security team [WinKernel] say:

> Between 2017 and mid 2018, this feature would have killed 49 MSRC cases that involved uninitialized struct data leaking across a trust boundary. It would have also mitigated a number of bugs involving uninitialized struct data being used directly.
> To date, we are seeing noise level performance regressions caused by this change. We accomplished this by improving the compilers ability to kill redundant stores. While everything is initialized at declaration, most of these initializations can be proven redundant and eliminated.

They use pure zero initialization, and claim to have taken the overheads down to within noise. They provide more details in [CppConMSRC] and [InitAll]. Don't just trust Microsoft's Windows security team though, here's one of the upstream Linux Kernel security developer asking for this [CLessDangerous], and Linus agreeing [Linus]. [LinuxExploits] is an overview of a real-world execution control exploit using an uninitialized stack variable on Linux. It's been proposed for GCC [init-local-vars] and LLVM [LocalInit] before.

In Chrome we find 1100+ security bugs for "use of uninitialized value" [ChromeUninit].

12% of all exploitable bugs in Android are of this kind according to [AndroidSec].

Kostya Serebryany has a very long list of issues caused by usage of uninitialized stack variables [KCC].

§ 4. Alternatives

When this is discussed, the discussion often goes to a few places.

First, people will suggest using tools such as memory sanitizer or valgrind. Yes they should, but doing so:

- Requires compiling everything in the process with memory sanitizer.
- Can't deploy in production.
- Only find that is executed in testing.
- 3× slower and memory-hungry.

The next suggestion is usually couldn't you just test / fuzz / code-review better or just write perfect code? However:

- We are (well, except the perfect code part).
- It's not sufficient, as evidenced by the exploits.
- Attackers find what programmers don't think of, and what fuzzers don't hit.

The annoyed suggester then says "couldn't you just use `-Werror=uninitialized` and fix everything it complains about?" This is similar to the [CoreGuidelines] recommendation. You are beginning to expect shortcoming, in this case:

- Too much code to change.
- Current `-Wuninitialized` is low quality, has false positives and false negatives. We could get better analysis if we had a compiler-frontend-based IR, but it still wouldn't be noise-free.
- Similarly, static analysis isn't smart enough.
- The value chosen to initialize doesn't necessarily make sense, meaning that code might move from unsafe to merely incorrect.
- Code doesn't express intent anymore, which leads readers to assume that the value is sensible when it might have simply been there to address the diagnostic.
- It prevents checkers (static analysis, sanitizers) from diagnosing code because it was given semantics which aren't intended.

Couldn't you just use definitive initialization in the language? For example as explained in [CppConDefinite] and [CppConComplexity]. This one is interesting. Languages such as C# and Swift allow uninitialized values, but only if the compiler can prove that they're not used uniitialized, otherwise they must be initialized. It's similar to enforcing `-Wuninitialized`, depending on what is standardized. We then have a choice: standardize a simple analysis (such as "must be initialized within the block or before it escapes") and risk needless initializations (with mitigations explained in the prior references), or standardize a more complex analysis. We would likely want to deploy a low-code-churn change, therefore a high-quality analysis. This would require that the committee standardize the analysis, lest compilers diverge. But a high-quality analysis tends to be complex, and change as it improves, which is not sensible to standardize. This would leave us standardizing a high-code-churn analysis with much more limited visibility. Further with definitive initialization, it's then unclear whether `int derp = 0;` lends meaning to `0`, or whether it's just there to shut that warning up.

An alternative for the `[[uninitialized]]` attribute is to use a keyword. We can argue about which keyword is right. However, maybe through [P0146r1], we'll eventually agree that `void` is the right name for "This variable is not a variable of honor... no highly esteemed deed is commemorated here... nothing valued is here." This approach meshes well with a definitive initialization approach because we would keep the semantics of "no value was intended on this variable".

One alternative that was suggested instead of zero-initialization is to value-initialize. This would invoke constructors and therefore be a no-go as a significant behavior change. That said, some form of "trivial enough" initialization might be an acceptable approach to increase correctness (not merely safety) for types for which zero isn't a valid value.

We could instead keep the out-of-band option that is already implemented in all major compilers. That out-of-band option is opt-in and keeps the status quo where "C++ is unsafe by default". Further, many compiler implementors are dissatisfied by this option because they believe it's effectively a language fork that users of the out-of-band option will come to rely on (this strategy is mitigated in MSVC by using a non-zero pattern in debug builds according to [InitAll]). These developers would rather see the language standardize the option officially.

Rather than zero-initializing, we could use another value. For example, the author implemented a mode where the fundamental types dictate the value used:

- Integral → repeated `0xAA`
- Pointers → repeated `0xAA`
- Floating-point → repeated `0xFF`

This is advantageous in 64-bit platforms because all pointer uses will trap with a very recognizable value, though most modern platforms will also trap for large offsets from the null pointer. Floating-point values are NaNs with payload, which propagate on floating-point operations and are therefore easy to root-cause. It is problematic for some integer values that represent sizes in a bounds check, because such a number is effectively larger that any bounds and would potentially permit out-of-bounds accesses that a zero initializetion would not.

Unfortunately, this scheme optimizes much more poorly and end up being a roughly 1.5% size and performance regression on some workloads according to [GoogleNumbers] and others.

Using repeated bytes for initialization is critical to effective code generation because instruction sets can materialize repeated byte patterns much more efficiently than any larger value. Any approach other than the above is bound to have a worst performance impact. That is why choosing implementation-defined values or runtime random values isn't a useful approach.

Another alternative is to guarantee that accessing uninitialized values is implementation-defined behavior, where implementations *must* choose to either, on a per-value basis:

- zero-initialize; or
- trap (we can argue separately as to whether "trap" is immediate termination, Itanium [NaT], or `std::terminate` or something else).

Implementations would then do their best to trap on uninitialized value access, but when they can't prove that an access is uninitialized through however complex optimization scheme, they would revert to zero-initialization. This is an interesting approach which Richard Smith has implemented a prototype of in [Trap]. However, the author's and others' experience is that such a change is rejected by teams because it cannot be deployed to codebases that seem to work just fine. The Committee might decide to go with this approach, and see whether the default in various implementations is "always zero" or "sometimes trap".

We could also standardized a hybrid approach that mixes some of the above, creating implementation-defined alternatives which are effectively different build modes that the standard allows. This is what [CarbonInit] does.

§ 5. Performance

Previous publications [Zeroing] have cited 2.7 to 4.5% averages. This was true because, in the author's experience implementing this in LLVM [LLVMJFB], the compiler simply didn't perform sufficient optimizations. Deploying this change required a variety of optimizations to remove needless work and reduce code size. These optimizations were generally useful for other code, not just automatic variable initialization.

As stated in the introduction, the performance impact is now negligible (less that 0.5% regression) to slightly positive (that is, some code gets *faster* by up to 1%). The code size impact is negligible (smaller than 0.5%).

Why do we sometimes observe a performance progression with zero-initialization? There are a few potential reasons:

- Zeroing a register or stack value can break dependencies, allowing more Instructions Per Cycles (IPC) in an out-of-order CPU by unblocking scheduling of instructions that were waiting on what seemed like a long critical path of instructions but were in fact only false-sharing registers. The register can be renamed to the hardware-internal zero register, and instructions scheduled.
- Zeroing of entire cache entries is magical, though for the stack this is rarely the case. There's questions of temporality, special instructions, and special logic to support zeroed cachelines.

Here are a few of the optimizations which have helped reduce overheads in LLVM:

- CodeGen: use non-zero `memset` for automatic variables D49771
- Merge clang's `isRepeatedBytePattern` with LLVM's `isBytewiseValue` D51751
- SelectionDAG: Reuse bigger constants in `memset` D53181

- ARM64: improve non-zero `memset` isel by ~2x D51706
- Double sign-extend stores not merged going to stack D54846 and D54847
- LoopUnroll: support loops w/ exiting headers & uncond latches D61962
- ConstantMerge: merge common initial sequences D50593
- IPO should track pointer values which are written to, enabling DSO DSO
- Missed dead store across basic blocks pr40527

This doesn't cover all relevant optimizations. Some optimizations will be architecture-specific, meaning that deploying this change to new architectures will require some optimization work to reduce cost.

New codebases using automatic variable initialization might trigger missed-optimizations in compilers and experience a performance or size regression. The wide deployment of the opt-in compiler flag means that this is unlikely, but were this to happen then a bug should be filed against the compiler to fix the missed optimization.

For example, here is Android's experience on performance costs [AndroidCosts].

Some of the performance costs are hidden by modern CPUs being heavily superscalar. Embedded CPUs are often in-order, code running on them might therefore have a performance regression that is roughly proportional to code size increases. Here too, compiler optimizations can dignificantly reduce these costs.

§ 6. Caveats

There are a few caveats to this proposal.

Making all automatic variables explicitly zero means that developers will come to rely on it. The current status-quo forces developers to express intent, and new code might decide not to do so and simply use the zero that they know to be there. This would then make it impossible to distinguish "purposeful use of the uninitialized zero" from "accidental use of the uninitialized zero". Tools such as memory sanitizers and valgrind would therefore be unable to diagnose correctness issues (but we would have removed the security issues). It should still be best practice to only assign a value to a variable when this value is meaningful, and only use an "uninitialized" value when meaning has been give to it.

This proposal will now mean that objects with dynamic storage duration are now uniquely different from all other storage duration objects, in that they alone are uninitialized. We could explore

guaranteeing initialization of these objects, but this is a much newer area of research and deployment as shown in [XNUHeap]. The strategy for automatically initializing objects of dynamic storage durations depend on a variety of factors, there's no agreed-upon one right solution for all codebases. Each implementation approach has tradeoffs that warrant more study.

Some types might not have a valid zero value, for example an enum might not assign meaning to zero. Initializing this type to zero is then safe, but not correct. However, C++ has constructors to deal with this type of issue.

For some types, the zero value might be the "dangerous" one, for example a null pointer might be a special sentinel value saying "has all priviledges" in a kernel. A concrete example is on Linux where `task->uid` of zero means `root`, uninitialized reads of zero have been the source of CVEs on Linux before.

Some people think that reading uninitialized stack variables is a good source of randomness to seed a random number generator, but these people are wrong [Randomness].

As an implementation concern for LLVM (and maybe others), not all variables of automatic storage duration are easy to properly zero-initialize. Variables declared in unreachable code, and used later, aren't currently initialized. This includes `goto`, Duff's device, other objectionable uses of `switch`. Such things should rather be a hard-error in any serious codebase.

`volatile` stack variables are weird. That's pre-existing, it's really the language's fault and this proposal keeps it weird. In LLVM, the author opted to initialize all `volatile` stack variables to zero. This is techically acceptable because they don't have any special hardware or external meaning, they're only used to block the optimizer. It's technically incorrect because the compiler shouldn't be performing extra accesses to `volatile` variables, but one could argue that the compiler is zero-initializing the storage right before the `volatile` is constructed. This would be a pointless discussion which the author is happy to have if the counterparty is buying.

§ 7. Wording

Wording for this proposal will be provided once an approach is agreed upon.

The author expects that the solution that is agreed upon maintains the property that:

- Taking the address, passing this address, and manipulating the address of data not explicitly initialized remains valid.
- `memcpy` of data not explicitly initialized (including padding) remains valid, but reading from the data or the `memcpy`d data is invalid.

§ References

§ Informative References

[AndroidCosts]

Alexander Potapenko. Fighting Uninitialized Memory in the Kernel. 2020. URL: https://clangbuiltlinux.github.io/CBL-meetup-2020-slides/glider/Fighting_uninitialized_memory_%40_CBL_Meetup_2020.pdf

[AndroidSec]

Nick Kralevich. How vulnerabilities help shape security features and mitigations in Android. 2016-08-04. URL: https://www.blackhat.com/docs/us-16/materials/us-16-Kralevich-The-Art-Of-Defense-How-Vulnerabilities-Help-Shape-Security-Features-And-Mitigations-In-Android.pdf

[AutoInitSummary]

Florian Hahn. [llvm-dev] RFC: Combining Annotation Metadata and Remarks. 2020-11-04. URL: https://lists.llvm.org/pipermail/llvm-dev/2020-November/146393.html

[CarbonInit]

Chandler Carruth. Carbon: Initialization of memory and variables. 2021-06-22. URL: https://github.com/carbon-language/carbon-lang/blob/trunk/proposals/p0257.md

[ChromeUninit]

multiple. Chromium code search for Use-of-uninitialized-value. multiple. URL: https://bugs.chromium.org/p/chromium/issues/list?can=1&q=Stability%3DMemory-MemorySanitizer+-status%3ADuplicate+-status%3AWontFix+Use-of-uninitialized-value

[CLessDangerous]

Kees Cook. Making C Less Dangerous. 2018-08-28. URL: https://outflux.net/slides/2018/lss/danger.pdf

[CoreGuidelines]

Herb Sutter; Bjarne Stroustrup. C++ Core Guidelines: ES.20: Always initialize an object. unknown. URL: https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#Res-always

[CppConComplexity]

Herb Sutter. Empirically Measuring, & Reducing, C++'s Accidental Complexity. 2020-10-11. URL: https://www.youtube.com/watch?v=6lurOCdaj0Y&t=971s

[CppConDefinite]

Herb Sutter. Can C++ be 10x Simpler & Safer?. 2022-11-13. URL: https://www.youtube.com/clip/UgkxpWxXNaoWvGqhlpSdWRQs0M3ayQcEhDoh

[CppConMSRC]

Joseph Bialek; Shayne Hiet-Block. Killing Uninitialized Memory: Protecting the OS Without Destroying Performance. 2019-10-13. URL: https://www.youtube.com/watch?v=rQWjF8NvqAU

[CWE-457]

MITRE. CWE-457: Use of Uninitialized Variable. 2006-07-19. URL: https://cwe.mitre.org/data/definitions/457.html

[GCCReview]

Qing Zhao. [RFC][patch for gcc12][version 1] add -ftrivial-auto-var-init and variable attribute "uninitialized" to gcc. 2021-02-18. URL: https://gcc.gnu.org/pipermail/gcc-patches/2021-February/565514.html

[GoogleNumbers]

Kostya Serebryany. [cfe-dev] [RFC] automatic variable initialization. 2019-01-16. URL: https://lists.llvm.org/pipermail/cfe-dev/2019-January/060878.html

[INIT-LOCAL-VARS]

Florian Weimer. -finit-local-vars option. 2014-06-06. URL: https://gcc.gnu.org/ml/gcc-patches/2014-06/msg00615.html

[InitAll]

Joseph Bialek. Solving Uninitialized Stack Memory on Windows. 2020-05-13. URL: https://msrc-blog.microsoft.com/2020/05/13/solving-uninitialized-stack-memory-on-windows/

[KCC]

Kostya Serebryany. [cfe-dev] [RFC] automatic variable initialization. 2018-11-15. URL: https://lists.llvm.org/pipermail/cfe-dev/2018-November/060177.html

[Linus]

Linus Torvalds. Re: [GIT PULL] meminit fix for v5.3-rc2. 2019-06-30. URL: https://lore.kernel.org/lkml/CAHk-=wgTM+cN7zyUZacGQDv3DuuoA4LORNPWgb1Y_Z1p4iedNQ@mail.gmail.com/

[LinuxExploits]

Kees Cook. Kernel Exploitation via Uninitialized Stack. 2019-08. URL: https://outflux.net/slides/2011/defcon/kernel-exploitation.pdf

[LinuxInfoleak]

Mathias Krause. Linux kernel: net - three info leaks in rtnl. 2013-03-19. URL: https://seclists.org/oss-sec/2013/q1/693

[LinuxUse]

Kangjie Lu; et al. Unleashing Use-Before-Initialization Vulnerabilities in the Linux Kernel Using Targeted Stack Spraying. 2017-03-01. URL: https://www-users.cs.umn.edu/~kjlu/papers/tss.pdf

[LLVMJFB]

JF Bastien. Mitigating Undefined Behavior. 2019-12-12. URL: https://www.youtube.com/watch?v=I-XUHPimq3o

[LLVMReview]

JF Bastien. Automatic variable initialization. 2018-11-15. URL: https://reviews.llvm.org/D54604

[LocalInit]

thestinger. add LocalInit sanitizer. 2018-06-22. URL: https://github.com/AndroidHardeningArchive/platform_external_clang/commit/776a0955ef6686d23a82d2e6a3cbd4a6a882c31c

[NaT]

Raymond Chen. Uninitialized garbage on ia64 can be deadly. 2004-01-19. URL: https://devblogs.microsoft.com/oldnewthing/20040119-00/?p=41003

[P0146r1]

Matt Calabrese. Regular Void. 11 February 2016. URL: https://wg21.link/p0146r1

[P0632r0]

Jonathan Müller. Proposal of [[uninitialized]] attribute. 19 January 2017. URL: https://wg21.link/p0632r0

[P2687r0]

Bjarne Stroustrup, Gabriel Dos Reis. Design Alternatives for Type-and-Resource Safe C++. 15 October 2022. URL: https://wg21.link/p2687r0

[Randomness]

Xi Wang. More randomness or less. 2012-06-25. URL: https://kqueue.org/blog/2012/06/25/more-randomness-or-less/

[Trap]

Richard Smith. [NOT FOR REVIEW] Experimental support for zero-or-trap behavior for uninitialized variables. 2020-05-01. URL: https://reviews.llvm.org/D79249

[WinKernel]

Joseph Bialek. Join the Windows kernel in wishing farewell to uninitialized plain-old-data structs on the stack. 2018-11-14. URL: https://twitter.com/JosephBialek/status/1062774315098112001

[XNUHeap]

Apple Security Engineering and Architecture. Towards the next generation of XNU memory safety: kalloc_type. 2022-10-22. URL: https://security.apple.com/blog/towards-the-next-generation-of-xnu-memory-safety/

[Zeroing]

Xi Yang; et al. Why Nothing Matters: The Impact of Zeroing. 2011-10. URL: https://users.elis.ugent.be/~jsartor/researchDocs/OOPSLA2011Zero-submit.pdf